Emory Hyper Community Cloud Cluster (C3) User Guide

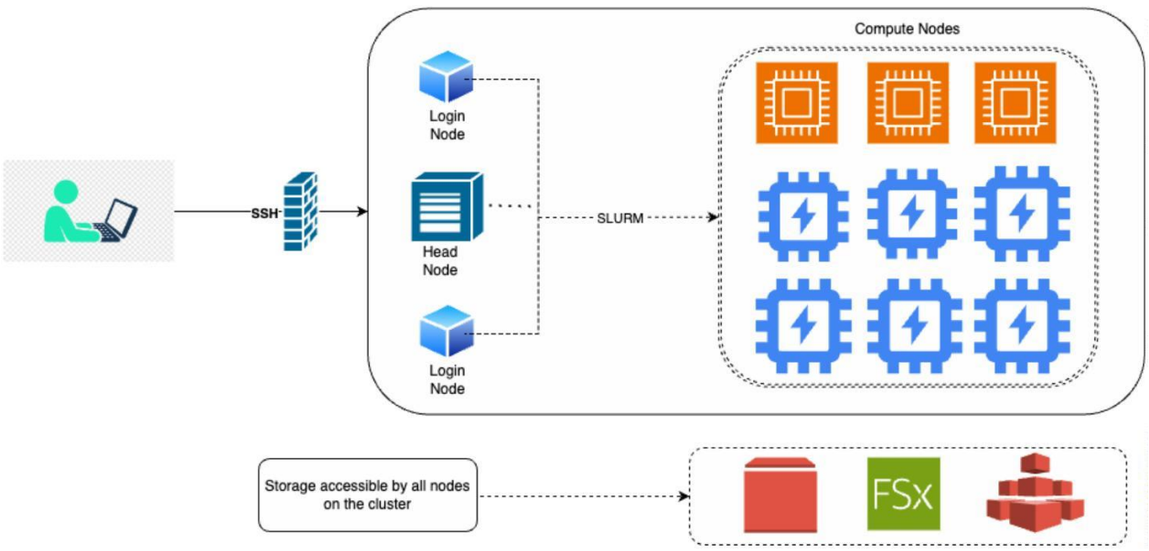
Table of Contents

Overview

The Emory Hybrid High-Performance Computing Platform for Education and Research Community Cloud Custer (HyPER C3) is a distributed computing system hosted on AWS at Emory. The cluster is designed to provide researchers with the computational resources they need to conduct research. HyperC3 cluster is HIPAA compliant. This means that the cluster is designed to protect the privacy and security of patient health information.

Architecture Diagram

This section describes the architecture of the HPC cloud cluster.



Logging in to the Cluster

This section provides instructions on how to login to the cluster.

(Note: Users accessing the cluster out of Emory Network should connect to Emory VPN)

Linux / Mac users: Terminal/ SHELL / iTerm

Windows users: cmd/Putty/MobaXterm

```
ssh <netid>@cirrostratus.it.emory.edu
```

(or)

```
ssh <netid>@ondemand.it.emory.edu
```

E.g.

```
ssh lganjicirrostratus.it.emory.edu
```

(or)

```
ssh lganjiondemand.it.emory.edu
```

(Note: Users are automatically routed to different login nodes based on traffic, which may sometimes result in a "Host Verification Error" or "Remote Host Identification Has Changed!" error. If you encounter this, follow any of the following methods and then run above login command again)

Method 1: Run the following steps and rerun above ssh login command

Step1: Download ssh_ca.pub key

Step2:

Mac/Linux Users:

```
echo "@cert-authority cirrostratus.it.emory.edu $(cat ssh_ca.pub)" >> ~/.ssh/known_hosts echo "@cert-authority ondemand.it.emory.edu $(cat ssh_ca.pub)" >> ~/.ssh/known_hosts
```

Windows Users:

```
type C:\path\to\ssh_ca.pub >> %USERPROFILE%\.ssh\known_hosts
```

(Note: Replace path above)

```
echo @cert-authority cirrostratus.it.emory.edu %USERPROFILE%\.ssh\known_hosts echo @cert-authority ondemand.it.emory.edu %USERPROFILE%\.ssh\known_hosts
```

Method 2: Run the following command and rerun above ssh login command (Not recommended)

```
ssh-keygen -R cirrostratus.it.emory.edu
```

General Setup

This section provides an overview of the cluster, including its hardware and software components.

1.Node:

A node is an individual computer or server within a cluster that works with other nodes to perform tasks and share resources HypeC3 cluster has 3 types of nodes:

a. Head Node

A head node in a cluster, also known as a master node, is a central server that manages and coordinates the operations of all other nodes in the cluster. Head Node in HyperC3 can only be accessed by admins and is restricted for general users.

b. Login Nodes

A login node in a cluster is a designated server that provides users with a point of entry to the cluster. There are two login nodes in HyperC3. When user's login, they will be logged into one of the two login nodes depending on the amount of traffic the node has.

c. Compute Nodes

Compute nodes in a cluster are servers specifically designated to perform the computational tasks and data processing jobs submitted by users. There are two types of compute nodes in HyperC3.

- **CPU nodes:**
  High processing power servers, grouped into partitions, handle generalpurpose and complex tasks. These nodes are designed to handle a wide variety of tasks efficiently, including single-threaded and multi-threaded operations. CPU's have higher clock speeds, but fewer cores compared to GPUs
- **GPU nodes:**
  Servers with attached GPUs, optimized for parallel processing tasks requiring high throughput. These nodes are used for tasks that benefit from the parallel processing capabilities of GPUs, such as machine learning, graphics rendering, and scientific computations. GPU's Can handle thousands of parallel threads, making them ideal for tasks like matrix multiplications and other data-parallel algorithms

2. Partitions:

A partition in a cluster refers to a logical grouping of nodes within the cluster that are designated for specific types of tasks or workloads. Partitions help in organizing the cluster's resources efficiently and can be used to manage and allocate resources based on user needs, workload types, or priority levels. The cluster currently has 11 partitions (OnDemand + Reserved). Among them the last 4 are backup partitions.

(Note: Backup partitions will be available for users only when the regular partitions are having issues)

***** Basic information on this Amazon ParallelCluster 3.10.1 of Emory University *****

Partitions	InstanceType	CapacityType	# of vCPUs	Memory of CPUs(GB)	# of GPUs	Memory of GPUs(GB)	Size of NVMe SSD storage(GB)	# of Compute Nodes
c64-m512	r7i.16xlarge	ONDEMAND	64	512	0	0	N/A	6
c128-m1024	r6id.32xlarge	ONDEMAND	128	1024	0	0	4x1900	1
l4-1-gm24-c16-m64	g6.4xlarge	ONDEMAND	16	64	1	24	1x250	2
l4-4-gm96-c48-m192	g6.12xlarge	ONDEMAND	48	192	4	96	4x940	3
l4-8-gm192-c192-m768	g6.48xlarge	ONDEMAND	192	768	8	192	8x940	1
l40s-8-gm384-c192-m1536	g6e.48xlarge	ONDEMAND	192	1536	8	384	1x7600	1
a100-8-gm320-c96-m1152	p4d.24xlarge	ALWAYS ON	96	1152	8	320	8x1000	1
a100-8-gm640-c96-m1152	p4de.24xlarge	ALWAYS ON	96	1152	8	640	8x1000	1
Disabled backup partitions:								
c64-m512-bk	r7i.16xlarge	ONDEMAND	64	512	0	0	N/A	6
c128-m1024-bk	r6id.32xlarge	ONDEMAND	128	1024	0	0	4x1900	1
l4-1-gm24-c16-m64-bk	g6.4xlarge	ONDEMAND	16	64	1	24	1x250	2
l4-4-gm96-c48-m192-bk	g6.12xlarge	ONDEMAND	48	192	4	96	4x940	3
l4-8-gm192-c192-m768-bk	g6.48xlarge	ONDEMAND	192	768	8	192	8x940	1

-> After a job has been allocated to an on demand compute node, it may experience a delayed start
-> During this time, the user may receive an email notification with a NODE_FAIL code.
-> The NODE_FAIL error can be ignored.

3. Slurm:

SLURM (Simple Linux Utility for Resource Management) is an open-source job scheduler used by many high-performance computing (HPC) clusters. It helps in allocating resources, managing job queues, and scheduling tasks efficiently across the cluster's compute resources.

Slurm contains many components in the user and admin level. The following are few user level components. A brief discussion of these components is provided in Job management section.

- **sinfo:** Displays the status of nodes and partitions
- **squeue:** Shows the status of jobs in the queue
- **sbatch:** Submits a job script for execution
- **srun:** Runs a command or script directly on the compute nodes
- **scancel:** Cancels a job
- **salloc:** Interactively allocate resources

4.Conda environments:

Using a conda virtual environment is recommended when installing software on the HPC cloud cluster. This will help ensure that the software is isolated from the rest of the system and will not conflict with other software installed on the cluster. For e.g.ma

To Create a conda environment named myenv

```
conda create -n myenv
```

To Create a conda environment with name myenv with python3.9

```
conda create -n myenv python=3.9
```

Conda environment named myenv, python version 3.9, and numpy, pandas packages

```
conda create -n myenv python=3.9 numpy pandas
```

List conda environments

```
conda env list
```

Remove conda environment

```
conda env remove --name myenv
```

More about conda environments can be found in Software management section.

(Note: Each user has a 50 GB storage limit in their home directory. If you reach this limit, you won't be able to install additional packages. To free up space, consider deleting unnecessary packages or using group storage for installing conda environments.)

Conda Environments can also be installed in scratch, group Storage and the environments created in group storage can be shared between the lab members.

E.g. `conda create --prefix=/group/PINETID-g00/envname python=3.8`

5.Shared Conda environments:

A list of popular packages/libraries are already installed in a shared folder that can be utilized by all the users using the cluster.

These can be found at /group/hpcsupport/env.

```
(base) [lganji@hyper-01-prod-ondemand-241-190 ~]$ ls /group/hpcsupport/env/
glibc  mxnet  mxnet-gpu  pytorch  pytorch-gpu  tensorflow  tensorflow-gpu  tf-gpu
```

To use these pre-installed conda environments in include the following command in your job script

```
conda activate /group/hpcsupport/env/envname
```

Account Management

1. Requesting a New Account

To begin using the cluster, the lead researcher, typically the Principal Investigator (PI), also referred to as the Sponsor User, must apply for an account. This can be done by completing the application form available on SharePoint [here]. Once the application is approved, the Sponsor User account will be created. Subsequently, each lab member must submit the same application form to gain access to the cluster.

2. User Account Expiration

User's accounts will become inactive when they leave Emory. Sponsor Users can monitor accounts to check for users who should no longer have access by requesting a user report from hpc.help@emory.edu.

Once granted access to a cluster, a user must log in within 60 days, or their account will be automatically inactivated. There is a 30-day grace period before automatic deactivation. Account data for inactive users is automatically deleted 90 days from their last login. This represents the 60-day account expiration plus the 30-day grace period. The account Sponsor will be notified before this happens.

Storage Types

The following section includes a brief description of cluster storage types for */users, /group*, and */scratch*, and their appropriate use cases.

/users

This is the default home directory for all users on the cluster. This is a personal directory used to store user-specific files such as configuration files, login scripts, and personal data. It is limited to 50GB and is purged when the user account becomes inactive and deactivated. Users should use the /scratch directory for data storage. Contents of /users directories are backed up nightly.

/group

Each faculty or departmental administrator who sponsors a group of cluster users is granted a default /group directory at /group/[netID]-g00. Access to this storage is granted through roles in MyNetID and can be administered by the sponsor, their defined proxy, or a member of the HPC support staff. Additional directories for projects that require data segregation may be requested.

The /group directory is a shared storage accessible to all group members. It is used to store files used by multiple users, such as data sets, software libraries, and documentation. This directory is permanent and has 1TB storage provided for free per group. Excess storage will be billed to the sponsor of the group. Users have read and write access to their own files unless otherwise specified. Users also have read-only access to their groups' files unless given specific permission.

If customized permission levels are needed among various team members or with collaborators, additional group share(s) can be requested by emailing hpc.help@emory.edu

Note that contents of /group directories are not backed up and cannot be restored administratively.

NOTE: We recommend running Python codes under the /users and **not** the /group directory for security purposes.

/scratch

Scratch storage makes use of FSx for Lustre storage, so it is the most performant type of storage available to users. Therefore, it is highly recommended that users store their input and output data in the /scratch directory. This directory is a permanent directory, and its contents will not be lost even if the cluster is destroyed. The /scratch directory is limited to 1TB per user. This quota may be temporarily increased up to 1.5Tif needed by sending a request to hpc.help@emory.edu. Note that contents in /scratch directories are not backed up and cannot be restored administratively so should only be used to hold data that can be easily re-created or backed up someplace else.

Additional Storage Type(s)

Mounting AWS S3 bucket to the Login Node

It is possible to mount an existing S3 bucket containing data for read-only usage or transfer of data into other more performant storage types. Submit request to *hpc.help@emory.edu* with name, User NetID, Sponsor Netid, reason, S3 bucket AWS account ID, S3 bucket name. The HPC support staff will work with sponsor and/or their proxies to build the needed AWS credentials and mount the S3 bucket based on the feasibility of the request.

Note that data from mounted S3 Buckets is only accessible via the Login Node and is not shared to the Compute Nodes.

Mounted S3 buckets will be mounted at a /datatransfer path on the Login Node. Full path will be provided to the sponsor once the mounting is complete.

Storage Quotas

Users can check remaining storage in a particular type of storage using the commands listed below (all commands should be run once logged onto the head node).

1. /users directory

```
du -sh
```

Relevant output:

```
(base) [lganji@hyper-01-prod-ondemand-241-187 ~]$ du -sh
31G     .
```

This shows 31 GB is used. The quota is 50 GB.

(Note: To get list of each file usage use `du -sh .[!.]* *)`

2. /scratch directory

```
lfs quota -u netID /scratch
```

(where netID is the logged in user)

Relevant output:

```
(base) [lganji@hyperprod-head-ip-10-66-241-209 ~]$ lfs quota -u lganji /scratch
Disk quotas for usr lganji (uid 7645):
     Filesystem  kbytes   quota   limit   grace   files   quota   limit   grace
       /scratch   28413       0 1000000000       -      64       0       0       -
```

This shows 28413 KB used out of a 1TB quota on /scratch storage.

3./group directory

/group directory storage is not quota-ed, however, the user is encouraged to monitor the storage as usage over 1TB will be charged to user/project speed types

```
du -sh /group/[GROUP DIRECTORY]
```

(where GROUP_DIRECTORY) is the project directory for shared storage

Relevant output:

```
[root@hyperprod-head-ip-10-66-241-209 group]# du -sh /group/ctsui2-g00
12K     /group/ctsui2-g00
```

This shows 12KB used in the shared */group/ctsui2-g00* directory.

NOTE: If you run into a storage quota in the */users* or */scratch* directories, you will get an error message stating, "Disk quota exceeded." Note that if you are in the middle of transferring a large file, it will write that file until you reach the quota and then stop, leaving you with a partial file that should be deleted.

General Job workflow

Small analysis that does not require large data input/output.

- Transfer input data from the source to */users*.
- Submit a job to perform the analysis and output to */users*.
- Once the job has been completed, transfer output from */users* to local or other storage outside of the cluster.

Analysis that requires space larger than /users

- Create a directory under */scratch*.
- Determine the input location as /users, */group* or */scratch*.
- Determine the output location as /users, /group or */scratch*.
- Transfer input data to the input location.

- Setup workflow to read from the input location, write temp files to */scratch* and write output to the output location.
- Submit a job to perform the analysis.
- Once the job has been completed, transfer output from the output location to local or other storage outside of the cluster.

Data transfer

This section provides instructions on how to transfer data to and from the cluster.

Data Transfer Methods overview

The following are some of the commonly used command-line tools that can be used to transfer files.

1. SCP

SCP stands for Secure Copy Protocol. It is a network protocol that securely copies files between a local host and a remote host, or between two remote hosts. SCP uses the Secure Shell (SSH) protocol to encrypt all data transferred, including the file contents and any passwords used for authentication. This is a very secure way to transfer files, even over public networks. It is a command line typically used to transfer files from one Linux to another.

In the following example, the file ***filename*** will be copied to/from the user's scratch directory on Cirrostratus using SCP.

From the local machine, open a terminal window and use SCP, where ***user_name*** is the Emory NetID.

- Local to Cirrostratus

```
scp filename
user_name@cirrostratus.it.emory.edu:/scratch/user_name/
```

- Cirrostratus to local

```
scp
user_name@cirrostratus.it.emory.edu:/scratch/user_name/filena
me .
```

To copy files/folder recursively use option -r

- Local to Cirrostratus

```
scp -r foldername
user_name@cirrostratus.it.emory.edu:/scratch/user_name/
```

- Cirrostratus to local

```
scp -r
```

```
user_name@cirrostratus.it.emory.edu:/scratch/user_name/folder
name .
```

2. RSYNC

Rsync, short for remote sync, is a file copying tool used to replicate files and directories between local and remote systems. *Rsync* is an especially useful tool for keeping local and remote datasets synchronized because *rsync* will only copy the differences between the source files and the existing files in the destination.

Rsync should be configured to use *SSH* as the remote shell to securely connect to Cirrostratus. Using *SSH* will provide encryption in transit which is required when transferring data to/from Cirrostratus. The example below shows how to ensure SSH is used.

In this example, the file filename will be copied to/from the user's scratch directory on Cirrostratus using *SCP*.

From the local machine, open a terminal window and use *rsync* to start a transfer.

- Local to Cirrostratus

```
rsync -e ssh filename
username@cirrostratus.it.emory.edu:/scratch/user_name/
```

- Cirrostratus to local

```
rsync -e ssh
user_name@cirrostratus.it.emory.edu:/scratch/user_name/filena
me .
```

3. SFTP

While SCP is a non-interactive way to copy files between two hosts, Secure File Transfer Protocol (*SFTP*) allows you to navigate and copy files interactively.

In the following example, the file ***filename*** will be copied to/from the user's scratch directory on Cirrostratus using SFTP.

From the local machine, open a terminal window and initiate an SFTP session.

- To initiate an SFTP session, type:

```
sftp user_name@cirrostratus.it.emory.edu
```

where **user_name** is the Emory NetID. There will be a prompt for password authentication.

- An SFTP prompt will be opened:

```
sftp>
```

- Navigate to the share directory:

```
sftp> cd /scratch/user_name
```

- Local to Cirrostratus

```
sftp> put filename
```

- Cirrostratus to local

```
sftp> get filename
```

- To exit the SFTP session:

```
sftp> quit
```

4. Globus Data Transfer

To setup Globus Transfer please submit a ticket to hpc.help@emory.edu

5. AWS CLI 2

AWS CLI 2 is a powerful tool that can be used to interact with AWS services from the command line. It can be used to transfer data in a variety of ways, and it is a valuable tool for anyone who works with AWS. This is available for users to transfer files between the head node and their S3 buckets.

- Log onto the cluster:

```
ssh netid@cirrostratus.it.emory.edu
```

Paste the aws access keys in the terminal.

(**NOTE**: To adhere to best practices, avoid using recursive copying with aws s3 cp. However, if necessary, run the provided script before executing aws s3 cp --recursive for copying between an S3 bucket and a cluster. This script configures various S3 operation settings)

```
#!/bin/bash

 #This command sets the maximum number of concurrent requests for S3 operations to 10 for the specified AWS profile

aws configure set --profile tki-aws-account-692-rhedcloud/RHEDcloudAdministratorRole s3.max_concurrent_requests 10

#This command sets the maximum queue size for S3 operations to 1000 for the specified AWS profile

aws configure set --profile tki-aws-account-692-rhedcloud/RHEDcloudAdministratorRole s3.max_queue_size 1000

#This command sets the multipart upload threshold for S3 operations to 64 megabytes (MB) for the specified AWS profile

aws configure set --profile tki-aws-account-692-rhedcloud/RHEDcloudAdministratorRole s3.multipart_threshold 64MB

#This command sets the multipart upload chunk size for S3 operations to 16 megabytes (MB) for the specified AWS profile

aws configure set --profile tki-aws-account-692-rhedcloud/RHEDcloudAdministratorRole s3.multipart_chunksize 16MB

#This command sets the maximum bandwidth for S3 operations to 100 megabytes per second (MB/s) for the specified AWS profile


aws configure set --profile tki-aws-account-692-rhedcloud/RHEDcloudAdministratorRole s3.max_bandwidth 100MB/s
```

- Cirrostratus to S3 bucket

```
aws s3 cp filename s3://mybucket/filename
```

- S3 bucket to Cirrostratus

```
aws s3 cp s3://mybucket/filename filename
```

Recommended Use Cases

- In general, *SCP* or *RSYNC* are the recommended approach for copying data to the HPC cluster because it is typically faster than *SFTP*.
- While Rsync provides an excellent copy tool, it really excels at synchronizing local and remote datasets.
- SFTP offers an interactive session that allows users to manage files, including viewing directories, creating folders, deleting, or renaming files, etc. This additional functionality might be helpful for users who need to search for a particular file.
- The *AWS CLI* (in combination with the *TKI* service) is the recommended approach for moving files to and from an S3 bucket.
- For larger data transfers (>1TB) OIT (Office of Information Technology) may be able to provide some other options like Globus Data Transfer– contact support at *hpc.help@emory.edu*

Job management (SLURM)

This section describes the most common SLURM terminologies and commands

1.Sinfo

sinfo is used to view partition and node information for a system running SLURM. An example of the output of the *sinfo* command can be seen below. The actual output might be different depending on new partitions that have been introduced or old ones that have been replaced/removed.

The output of the *sinfo* command lists each partition in the cluster and details like the availability status, time limit, number of nodes, state of nodes and list of node names associated with each partition.

The naming convention of the non-GPU partitions is :

```
c<no. of vCPU cores>-m<amount of memory>
```

The naming convention of the GPU partitions is:

```
<name of the GPU>-<no. of GPUs>-gm<amount of GPU memory>-
c<no. of vCPU cores>-m<amount of memory>
```

```
(base) [lganji@hyper-01-prod-ondemand-241-190 ~]$ sinfo
PARTITION                AVAIL  TIMELIMIT  NODES  STATE NODELIST
c64-m512*                   up 7-00:00:00      3  idle~ c64-m512-dy-r7i-16xlarge-[1,5-6]
c64-m512*                   up 7-00:00:00      3    mix c64-m512-dy-r7i-16xlarge-[2-4]
c128-m1024                  up 7-00:00:00      1    mix c128-m1024-dy-r6id-32xlarge-1
l4-1-gm24-c16-m64           up 7-00:00:00      2  alloc l4-1-gm24-c16-m64-dy-g6-4xlarge-[1-2]
l4-4-gm96-c48-m192          up 7-00:00:00      3  idle~ l4-4-gm96-c48-m192-dy-g6-12xlarge-[1-3]
l4-8-gm192-c192-m768        up 7-00:00:00      1    mix l4-8-gm192-c192-m768-dy-g6-48xlarge-1
l40s-8-gm384-c192-m1536     up 7-00:00:00      1    mix l40s-8-gm384-c192-m1536-dy-g6e-48xlarge-1
a100-8-gm320-c96-m1152      up 7-00:00:00      1    mix a100-8-gm320-c96-m1152-st-p4d-24xlarge-1
a100-8-gm640-c96-m1152      up 7-00:00:00      1    mix a100-8-gm640-c96-m1152-st-p4de-24xlarge-1
c64-m512-bk              drain 7-00:00:00      6  idle~ c64-m512-bk-dy-r7i-16xlarge-[1-6]
c128-m1024-bk            drain 7-00:00:00      1  idle~ c128-m1024-bk-dy-r6id-32xlarge-1
l4-1-gm24-c16-m64-bk     drain 7-00:00:00      2  idle~ l4-1-gm24-c16-m64-bk-dy-g6-4xlarge-[1-2]
l4-4-gm96-c48-m192-bk    drain 7-00:00:00      3  idle~ l4-4-gm96-c48-m192-bk-dy-g6-12xlarge-[1-3]
l4-8-gm192-c192-m768-bk drain 7-00:00:00      1  idle~ l4-8-gm192-c192-m768-bk-dy-g6-48xlarge-1
```

The commonly used options with *sinfo* are as follows:

- *-a, --all*: Displays information about all partitions. This causes information to be displayed about partitions that are configured as hidden and partitions that are unavailable to the user's group.
- *--help*: Prints a message describing all *sinfo* options.
- *-l, --long*: reports more of the available information for the selected jobs or job steps, subject to any constraints specified.

An example of the output of the *sinfo* command with the *--long* option can be seen below. The actual output might be different depending on new partitions that have been introduced or old ones that have been replaced/removed. In addition to the details seen in sinfo output, the *--long* option also provides details such as job size, is only root user allowed to initiate jobs, "*yes*" or "*no*", whether jobs may oversubscribe compute resources and groups which may use the nodes, STATE of the nodes.

```
(base) [lganji@hyper-01-prod-comp-di-0241-216 ~]$ sinfo -l
Wed Dec 11 22:20:26 2024
PARTITION                 AVAIL  TIMELIMIT   JOB_SIZE ROOT OVERSUBS     GROUPS  NODES       STATE RESERVATION NODELIST
c64-m512*                    up 7-00:00:00 1-infinite   no       NO        all      5       mixed             c64-m512-dy-r6id-16xlarge-[1-5]
c96-m768                     up 7-00:00:00 1-infinite   no       NO        all      1       mixed             c96-m768-st-r6id-24xlarge-1
a10g-1-gm24-c32-m128         up 7-00:00:00 1-infinite   no       NO        all      2       idle~             a10g-1-gm24-c32-m128-dy-g5-8xlarge-[1-2]
a10g-4-gm96-c48-m192         up 7-00:00:00 1-infinite   no       NO        all      2       mixed             a10g-4-gm96-c48-m192-dy-g5-12xlarge-[1-2]
a10g-4-gm96-c48-m192         up 7-00:00:00 1-infinite   no       NO        all      1   allocated             a10g-4-gm96-c48-m192-dy-g5-12xlarge-3
a10g-8-gm192-c192-m768       up 7-00:00:00 1-infinite   no       NO        all      2       mixed             a10g-8-gm192-c192-m768-dy-g5-48xlarge-[1-2]
a100-8-gm320-c96-m1152       up 7-00:00:00 1-infinite   no       NO        all      2       mixed             a100-8-gm320-c96-m1152-st-p4d-24xlarge-[1-2]
a100-8-gm640-c96-m1152       up 7-00:00:00 1-infinite   no       NO        all      1       mixed             a100-8-gm640-c96-m1152-st-p4de-24xlarge-1
c64-m512-bk                drain 7-00:00:00 1-infinite   no       NO        all      5       idle~             c64-m512-bk-dy-r6id-16xlarge-[1-5]
a10g-1-gm24-c32-m128-bk    drain 7-00:00:00 1-infinite   no       NO        all      2       idle~             a10g-1-gm24-c32-m128-bk-dy-g5-8xlarge-[1-2]
a10g-4-gm96-c48-m192-bk    drain 7-00:00:00 1-infinite   no       NO        all      3       idle~             a10g-4-gm96-c48-m192-bk-dy-g5-12xlarge-[1-3]
a10g-8-gm192-c192-m768-bk  drain 7-00:00:00 1-infinite   no       NO        all      2       idle~             a10g-8-gm192-c192-m768-bk-dy-g5-48xlarge-[1-2]
```

More detailed information on sinfo and its options can be found on - https://slurm.schedmd.com/sinfo.html

2.squeue

Squeue is used to view job and job step information for jobs managed by SLURM. It's used to view information about jobs located in the SLURM scheduling queue.

The commonly used options with squeue are as follows:

- -a, --all: Displays information about jobs and job steps in all partitions. This causes information to be displayed about partitions that are configured as hidden, partitions that are unavailable to a user's group, and federated jobs that are in a "revoked" state.
- --help: Prints a help message describing all options squeue.
- -l, --long: Reports more of the available information for the selected jobs or job steps, subject to any constraints specified.

--me, -u userid: Displays only jobs that are submitted by the user

3.Sbatch

sbatch submits a batch script to SLURM. The batch script may be given to *sbatch* through a file name on the command line, or if no file name is specified, *sbatch* will read in a script from standard input. The batch script is not necessarily granted resources immediately, it may sit in the queue of pending jobs for some time before its required resources become available.

The batch script may contain options preceded with "*#SBATCH*" before any executable commands in the script. *sbatch* will stop processing further *#SBATCH* directives once the first non-comment non-whitespace line has been reached in the script.

An example of a batch script with *#SBATCH* executable commands is shown below:

NOTE: Account, Partition, Memory, Time, GPU are required parameters. Output file will be *job-name* + job number unless otherwise requested in your script.

```
#!/bin/bash
#SBATCH --job-name=test_a10g-1_partition      # Name of the job
#SBATCH --account=general      # Use account='a100v100' for a100 gpus
#SBATCH --nodes=1              # Number of nodes requested
#SBATCH --ntasks=1             # Number of tasks
```

```
#SBATCH --partition=a10g-1-gm24-c32-m128      # Name of the partition
#SBATCH --time=01:00:00        #default time is 3-00:00:00; max 7-00:00:00
#SBATCH --output=%x_%j.out     # Name of the output file
#SBATCH --error=%x %j.err      # Name of the error file #SBATCH
--gpus=1              # Enter no.of gpus needed
#SBATCH --mem=2G               # Memory Needed
#SBATCH --mail-type=begin      # send mail when job begins
#SBATCH --mail-type=end        # send mail when job ends
#SBATCH --mail-type=fail       # send mail if job fails
#SBATCH --mail-user=lganji@emory.edu     # Replace with your mailid
conda init bash > /dev/null 2>&1     #initializes Conda for Bash shell
source ~/.bashrc                                                      #
sources ~/.bashrc file
conda activate torch-env # replace torch-env with your conda env
python sample a10g-1-gm24-c32-m128.py # replace with your script
```

(Note: In the slurm script even if you exclude --account it will automatically use account
'a100v100' for a100 partitions and general for other partitions)

This SLURM job requires 1 vCPU, 1 GPU, and 2GB Memory. This job will be submitted to the a10g-1-gm24-c32-m128 partition as specified in the script. This job is requesting for 1 node, 1 task per node and 1 vCPU per task. By default 1 vCPU per task is requested. However, if the job supports multi-threading, cpu-per-task can be used to request more than 1. For ensuring that the job script runs successfully it is especially important to specify the correct number of vCPUs and GPUs in the script. Refer to the banner below for information about the different partitions and their capacities to ensure that the job script adheres to these specified requirements and limits. Additional example scripts are available at https://github.service.emory.edu/LITS/hyper-user-scripts . For any questions related to the script or partitions and their capacities, please reach out to the support team using the email address - hpc.help@emory.edu

```
**********************************************************************************************************************************
**********************************************************************************************************************************
**                          ***** Basic information on this Amazon ParallelCluster 3.10.1 of Emory University *****                 **
**                                                                                                                                **
** Partitions                InstanceType       CapacityType  # of vCPUs  Memory of CPUs(GB)  # of GPUs Memory of GPUs(GB)   Size of NVMe SSD storage(GB)  # of Compute Nodes **
** ----------------------    -------------      ------------  ----------  ----------------    --------  -------------      ----------------------------  ------------------ **
** c64-m512                  r7i.16xlarge       ONDEMAND      64          512                 0         0                  N/A                           6                  **
** c128-m1024                r6id.32xlarge      ONDEMAND      128         1024                0         0                  4x1900                        1                  **
** l4-1-gm24-c16-m64         g6.4xlarge         ONDEMAND      16          64                  1         24                 1x250                         2                  **
** l4-4-gm96-c48-m192        g6.12xlarge        ONDEMAND      48          192                 4         96                 4x940                         3                  **
** l4-8-gm192-c192-m768      g6.48xlarge        ONDEMAND      192         768                 8         192                8x940                         1                  **
** l40s-8-gm384-c192-m1536   g6e.48xlarge       ONDEMAND      192         1536                8         384                1x7600                        1                  **
** a100-8-gm320-c96-m1152    p4d.24xlarge       ALWAYS ON     96          1152                8         320                8x1000                        1                  **
** a100-8-gm640-c96-m1152    p4de.24xlarge      ALWAYS ON     96          1152                8         640                8x1000                        1                  **
**                                                                                                                                **
** Disabled backup partitions:                                                                                                    **
**                                                                                                                                **
** c64-m512-bk                  r7i.16xlarge    ONDEMAND      64          512                 0         0                  N/A                           6                  **
** c128-m1024-bk                r6id.32xlarge   ONDEMAND      128         1024                0         0                  4x1900                        1                  **
** l4-1-gm24-c16-m64-bk         g6.4xlarge      ONDEMAND      16          64                  1         24                 1x250                         2                  **
** l4-4-gm96-c48-m192-bk        g6.12xlarge     ONDEMAND      48          192                 4         96                 4x940                         3                  **
** l4-8-gm192-c192-m768-bk      g6.48xlarge     ONDEMAND      192         768                 8         192                8x940                         1                  **
** ----------------------    -------------      ------------  ----------  ----------------    --------  -------------      ----------------------------  ------------------ **
**                                                                                                                                **
**    -> After a job has been allocated to an on demand compute node, it may experience a delayed start                          **
**    -> During this time, the user may receive an email notification with a NODE_FAIL code.                                      **
**    -> The NODE_FAIL error can be ignored.                                                                                      **
**********************************************************************************************************************************
**********************************************************************************************************************************
```

When the job allocation is finally granted for the batch script, SLURM runs a single copy of the batch script on the first node in the set of allocated nodes.

An example of running a batch script through a file is shown below. The squeue command output shows that the batch script was successfully submitted to the queue and has a state code of CF = Configuring.

```
(base) [lganji@hyperprod-head-ip-10-66-241-209 testjobs]$ sbatch test_a10g-1-gm24-c32-m128_partition.sh
sbatch: Your job is submitted to default account: general
sbatch: slurm_job_submit: initialized
Submitted batch job 81671
(base) [lganji@hyperprod-head-ip-10-66-241-209 testjobs]$ squeue
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
             81671 a10g-1-gm test_a10   lganji CF       0:06      1 a10g-1-gm24-c32-m128-dy-g5-8xlarge-1
```

4.squeue

When the compute node is booted and ready to run a job the state of the job changes to Running and can be monitored using squeue.

```
(base) [lganji@hyperprod-head-ip-10-66-241-209 testjobs]$ squeue
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
             81671 a10g-1-gm test_a10   lganji  R       0:46      1 a10g-1-gm24-c32-m128-dy-g5-8xlarge-1
```

For more information about the policy in places governing the resources and priority of the jobs, refer to the Compute Policy section in Terms and Policy

1. Script Path Resolution

- The batch script is resolved in the following order:
- If script starts with ".", then path is constructed as: current working directory /script.
- If script starts with a "/", then path is considered absolute.
- If script is in current working directory.
- If script can be resolved through PATH.

The current working directory is the calling process working directory unless the --chdir argument is passed, which will override the current working directory. The commonly used options with sbatch are as follows:

- -N, --nodes=<minnodes>[-maxnodes]: Request that a minimum of minnodes nodes be allocated to this job. A maximum node count may also be specified with maxnodes. If only one number is specified, this is used as both the minimum and maximum node count. The partition's node limits supersede those of the job. If a job's node limits are outside of the range permitted for its associated partition, the job will be left in a PENDING state. This permits execution later when the partition limit is changed. If a job node limit exceeds the number of nodes configured in the partition, the job will be rejected. Note that the environment variable SLURM_JOB_NUM_NODES will be set to the count of nodes actually allocated to the job.

- --ntasks-per-node=<ntasks>: Request that ntasks be invoked on each node. If used with the --ntasks option, the --ntasks option will take precedence and the --ntasksper-node will be treated as a maximum count of tasks per node. Meant to be used with the --nodes option. This is related to --cpus-per-task=ncpus but does not require knowledge of the actual number of cpus on

each node. In some cases, it is more convenient to be able to request that no more than a specific number of tasks be invoked on each node.

More detailed information on *sbatch* and its options can be found on - https://slurm.schedmd.com/sbatch.html

2.Job State Codes

Jobs typically pass through several states during their execution. The typical states are PENDING, CONFIGURING, RUNNING, SUSPENDED, COMPLETING, CANCELLED and COMPLETED. An explanation of each state follows.

- PD PENDING: Job is awaiting resource allocation.
- CF CONFIGURING: Job has been allocated resources but are waiting for them to become ready for use (e.g., booting).
- R RUNNING: Job currently has an allocation.
- S SUSPENDED: Job has an allocation, but execution has been suspended and CPUs have been released for other jobs.
- CG COMPLETING: Job is in the process of completing. Some processes on some nodes may still be active.
- CA CANCELLED: Job was explicitly cancelled by the user or system administrator. The job may or may not have been initiated.
- CD COMPLETED: job has terminated all processes on all nodes with an exit code of zero.

An example of the output of the *squeue* command with a job with state code PD = Pending, CF = CONFIGURING can be seen below:

Example 1

```
(base) [lganji@hyperprod-head-ip-10-66-241-209 JOBS]$ squeue
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
             81668 a100-8-gm test_a10   lganji PD       0:00      1 (BeginTime)
             81667 a10g-1-gm test_a10   lganji CF       1:44      1 a10g-1-gm24-c32-m128-dy-g5-8xlarge-1
             81665  c64-m512 test_c64   lganji CF       2:42      1 c64-m512-dy-r6id-16xlarge-1
             81666  c96-m768 test_c96   lganji CF       2:04      1 c96-m768-dy-r6id-24xlarge-1
```

Example 2

```
(base) [lganji@hyperprod-head-ip-10-66-241-209 JOBS]$ squeue -l
Tue Mar 12 00:43:59 2024
             JOBID PARTITION     NAME     USER    STATE       TIME TIME_LIMI  NODES NODELIST(REASON)
             81668 a100-8-gm test_a10   lganji  PENDING       0:00 7-00:00:00      1 (BeginTime)
             81667 a10g-1-gm test_a10   lganji CONFIGUR       1:39 7-00:00:00      1 a10g-1-gm24-c32-m128-dy-g5-8xlarge-1
             81665  c64-m512 test_c64   lganji CONFIGUR       2:37 7-00:00:00      1 c64-m512-dy-r6id-16xlarge-1
             81666  c96-m768 test_c96   lganji CONFIGUR       1:59 7-00:00:00      1 c96-m768-dy-r6id-24xlarge-1
```

More detailed information on squeue and its options can be found on - https://slurm.schedmd.com/squeue.html

5.Srun

srun runs a parallel job on cluster managed by SLURM. If necessary, srun will first create a resource allocation in which to run the parallel job. srun will submit the job request to the SLURM job controller, then initiate all processes on the remote nodes. If the request cannot be met immediately, *srun* will block until the resources are free to run the job.

When initiating remote processes srun will propagate the current working directory, unless --chdir=<path> is specified, in which case path will become the working directory for the remote processes.

An example of executing srun with hostname command can be seen below:

- ***srun* with one job step:**

```
#!/bin/bash

#SBATCH --nodes=1

#SBATCH --ntasks=1

#SBATCH --
time=02:00:00

#SBATCH --gpus=1

#SBATCH --mem=2G

#SBATCH --partition=a100-8-gm320-c96-m1152


conda init bash >/dev/null 2>&1 source

~/.bashrc

conda activate tensorflow_python39 sun
testGPU.py
```

- ***srun* with two job steps:**

```
#!/bin/bash

#SBATCH --nodes=1

#SBATCH --ntasks=1

#SBATCH --time=02:00:00

#SBATCH --gpus=1

#SBATCH --mem=2G

#SBATCH --partition=a100-8-gm320-c96-m1152


conda init bash >/dev/null 2>&1 source

~/.bashrc

conda activate tensorflow_python39
sun testGPU.py & srun mnist.py &
```

More detailed information on *srun* and its options can be found on - https://slurm.schedmd.com/srun.html

- **srun to login to compute nodes using jobid:**

Ssh into compute nodes is not allowed in HyperC3 cluster. But there is a way to login to the compute nodes using your jobid.

```
srun --jobid jobid --pty --preserve-env $SHELL
```

```
(base) [lganji@hyperprod-head-ip-10-66-241-163 ~]$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
             85515 a10g-1-gm torch-te   lganji  R      12:51      1 a10g-1-gm24-c32-m128-dy-g5-8xlarge-1
(base) [lganji@hyperprod-head-ip-10-66-241-163 ~]$
(base) [lganji@hyperprod-head-ip-10-66-241-163 ~]$
(base) [lganji@hyperprod-head-ip-10-66-241-163 ~]$ srun --jobid 85515 --pty --preserve-env $SHELL
 (base) [lganji@hyperprod-compute-ip-10-66-241-48 ~]$
```

In the above you can see using srun we are able to login to the compute node.

6.Scancel

scancel is used to signal or cancel jobs, job arrays or job steps. An arbitrary number of jobs or job steps may be signaled using job specification filters or a space separated list of specific job and/or job step IDs. If the job ID of a job array is specified with an array ID value, then only that job array element will be cancelled. If the job ID of a job array is specified without an array ID value, then all job array elements will be cancelled.

While a heterogeneous job is in a PENDING state, only the entire job can be cancelled rather than its individual components. A request to cancel an individual component of a heterogeneous job while in a PENDING state will return an error. After the job has begun execution, an individual component can be cancelled.

A job or job step can only be signaled by the owner of that job or user root. If an attempt is made by an unauthorized user to signal a job or job step, an error message will be printed, and the job will not be signaled.

An example of *scancel* with JOB ID as an argument is shown below. Once the job is cancelled it no longer shows up in the squeue output.

```
(base) [zwang28@ip-10-66-241-165 scripts]$ sbatch mnist_c32-m64.sh
sbatch: Not GPU job!
sbatch: slurm_job_submit: initialized
Submitted batch job 218
(base) [zwang28@ip-10-66-241-165 scripts]$
(base) [zwang28@ip-10-66-241-165 scripts]$ squeue
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
               218   c32-m64 mnist_on  zwang28 CF       0:03      1 c32-m64-dy-c6i-8xlarge-1
(base) [zwang28@ip-10-66-241-165 scripts]$
(base) [zwang28@ip-10-66-241-165 scripts]$ scancel 218
(base) [zwang28@ip-10-66-241-165 scripts]$
(base) [zwang28@ip-10-66-241-165 scripts]$ squeue
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
```

More detailed information on *scancel* and its options can be found on - https://slurm.schedmd.com/scancel.html

7.Scontrol

At the user level, `scontrol`is mainly useful for checking job details, modifying job properties and monitoring cluster resources. Here's how you can use `scontrol` effectively as a SLURM user

Check Job Information:

```
scontrol show job <job_id>
```

E.g.:

```
(base) [lganji@hyper-01-prod-ondemand-241-190 handsonscripts]$ scontrol show job 262290
JobId=262290 JobName=myfirstjob
   UserId=lganji(7645) GroupId=defaultLDAP(88) MCS_label=N/A
   Priority=255 Nice=0 Account=general QOS=normal
   JobState=RUNNING Reason=None Dependency=(null)
   Requeue=1 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=0:0
   RunTime=00:00:08 TimeLimit=3-00:00:00 TimeMin=N/A
   SubmitTime=2025-04-03T03:27:18 EligibleTime=2025-04-03T03:27:18
   AccrueTime=2025-04-03T03:27:18
   StartTime=2025-04-03T03:27:18 EndTime=2025-04-06T03:27:18 Deadline=N/A
   PreemptEligibleTime=2025-04-03T03:27:18 PreemptTime=None
   SuspendTime=None SecsPreSuspend=0 LastSchedEval=2025-04-03T03:27:18 Scheduler=Main
   Partition=c64-m512 AllocNode:Sid=ip-10-66-241-190:1514501
   ReqNodeList=(null) ExcNodeList=(null)
   NodeList=c64-m512-dy-r7i-16xlarge-3
   BatchHost=c64-m512-dy-r7i-16xlarge-3
   NumNodes=1 NumCPUs=1 NumTasks=1 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
   ReqTRES=cpu=1,mem=1G,node=1,billing=1
   AllocTRES=cpu=1,mem=1G,node=1,billing=1
   Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
   MinCPUsNode=1 MinMemoryNode=1G MinTmpDiskNode=0
   Features=(null) DelayBoot=00:00:00
   OverSubscribe=OK Contiguous=0 Licenses=(null) Network=(null)
   Command=/users/lganji/handsonscripts/slurmscript.sh
   WorkDir=/users/lganji/handsonscripts
   StdErr=/users/lganji/handsonscripts/myfirstjob_262290.err
   StdIn=/dev/null
   StdOut=/users/lganji/handsonscripts/myfirstjob_262290.out
   Power=
```

Check Node information

```
scontrol show node <nodename>
```

E.g.:

```
(base) [lganji@hyper-01-prod-ondemand-241-190 ~]$ scontrol show node a100-8-gm320-c96-m1152-st-p4d-24xlarge-1
NodeName=a100-8-gm320-c96-m1152-st-p4d-24xlarge-1 Arch=x86_64 CoresPerSocket=1
   CPUAlloc=10 CPUEfctv=96 CPUTot=96 CPULoad=140.57
   AvailableFeatures=static,p4d.24xlarge,p4d-24xlarge,gpu
   ActiveFeatures=static,p4d.24xlarge,p4d-24xlarge,gpu
   Gres=gpu:a100:8
   NodeAddr=10.66.241.112 NodeHostName=hyper-01-prod-comp-di-0241-112 Version=23.11.7
   OS=Linux 6.1.94-99.176.amzn2023.x86_64 #1 SMP PREEMPT_DYNAMIC Tue Jun 18 14:57:56 UTC 2024
   RealMemory=1120665 AllocMem=1048576 FreeMem=1074837 Sockets=48 Boards=1
   State=MIXED+CLOUD ThreadsPerCore=2 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
   Partitions=a100-8-gm320-c96-m1152
   BootTime=2025-03-31T22:06:49 SlurmdStartTime=2025-03-31T22:15:59
   LastBusyTime=2025-04-01T15:06:21 ResumeAfterTime=None
   CfgTRES=cpu=96,mem=1120665M,billing=96,gres/gpu=8,gres/gpu:a100=8
   AllocTRES=cpu=10,mem=1T,gres/gpu=5,gres/gpu:a100=5
   CapWatts=n/a
   CurrentWatts=0 AveWatts=0
   ExtSensorsJoules=n/a ExtSensorsWatts=0 ExtSensorsTemp=n/a
```

Check Partition Information

```
scontrol show partition <partitionname>
```

```
(base) [lganji@hyper-01-prod-ondemand-241-190 ~]$ scontrol show partition a100-8-gm320-c96-m1152
PartitionName=a100-8-gm320-c96-m1152
   AllowGroups=ALL AllowAccounts=ALL AllowQos=ALL
   AllocNodes=ALL Default=NO QoS=N/A
   DefaultTime=3-00:00:00 DisableRootJobs=NO ExclusiveUser=NO GraceTime=0 Hidden=NO
   MaxNodes=UNLIMITED MaxTime=7-00:00:00 MinNodes=0 LLN=NO MaxCPUsPerNode=UNLIMITED MaxCPUsPerSocket=UNLIMITED
   NodeSets=a100-8-gm320-c96-m1152_nodes
   Nodes=a100-8-gm320-c96-m1152-st-p4d-24xlarge-1
   PriorityJobFactor=1 PriorityTier=1 RootOnly=NO ReqResv=NO OverSubscribe=NO
   OverTimeLimit=NONE PreemptMode=REQUEUE
   State=UP TotalCPUs=96 TotalNodes=1 SelectTypeParameters=NONE
   JobDefaults=(null)
   DefMemPerNode=UNLIMITED MaxMemPerNode=UNLIMITED
   TRES=cpu=96,mem=1120665M,node=1,billing=96,gres/gpu=8,gres/gpu:a100=8
   ResumeTimeout=GLOBAL SuspendTimeout=GLOBAL SuspendTime=GLOBAL PowerDownOnIdle=NO
```

Monitoring GPU Utilization:

You can monitor gpu utilization using nvidia-smi command.

Once you login to the compute node using srun and jobid use the *nvidia-smi* command to track gpu utilization.

```
(base) [lganji@hyperdev-compute-ip-10-66-251-248 gpu-burn]$ nvidia-smi
Thu Mar 14 15:44:51 2024
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 470.182.03   Driver Version: 470.182.03   CUDA Version: 11.4     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla T4            On   | 00000000:00:1E.0 Off |                    0 |
| N/A   43C    P0    70W /  70W |  13609MiB / 15109MiB |    100%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|    0   N/A  N/A     14987      C   ./gpu_burn                      13606MiB |
+-----------------------------------------------------------------------------+
```

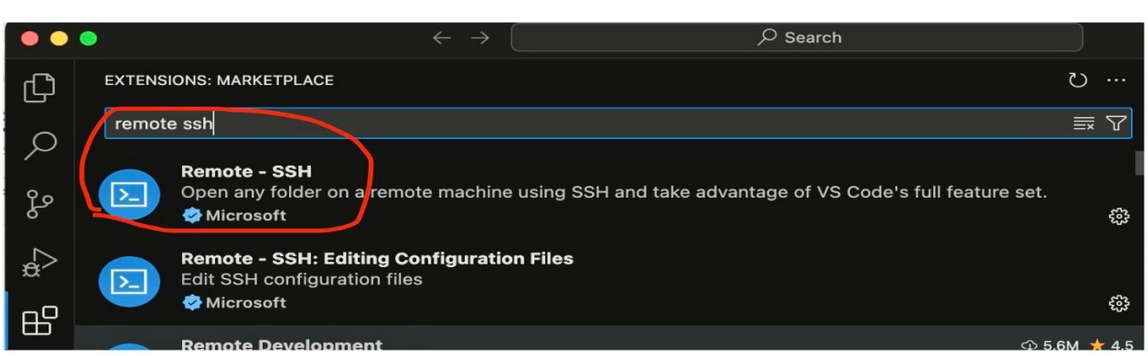
In the above image, you can notice all the details about the GPU and its utilization.

IDE Tools:

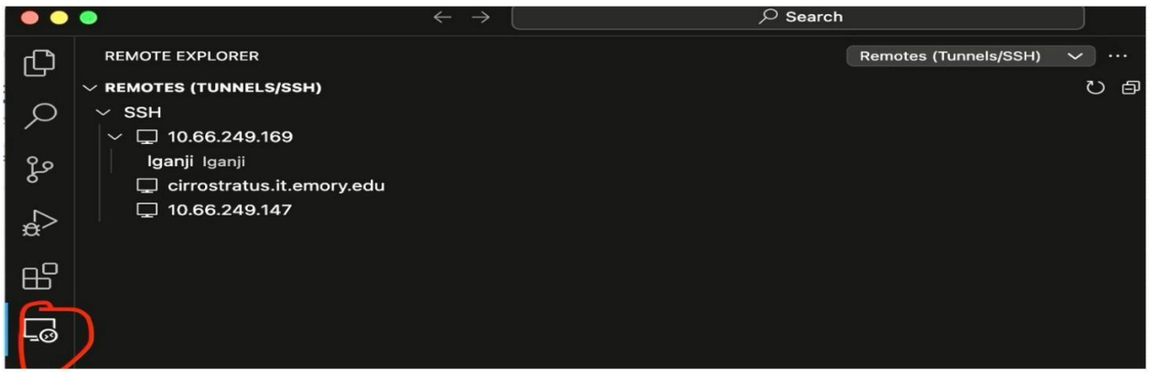
VSCode

Step1: Install VSCode Version >= 1.89.1

Step2: Install remote ssh extension



Step3: Click on Remote Explorer



Step4: Login to the server

Best Practices and Limitations for Environment Setup

- ## Avoid installing Jupyter Notebook

Installing JupyterNotebook has been restricted in the current upgrade. Installing resource intensive software like JupyterNotebook can monopolize system resources, potentially impacting the performance of critical cluster management tasks and user interactions.

- ## Using VSCode only for Editing Purpose

VSCode can be installed on the login node for accessing the files and also users can use the VScode terminal to submit jobs.

- ## Avoid running jobs on Login Node

In some scenarios, users run heavy workloads or scripts on the head node, which might increase the load on the head node and could lead to high CPU utilization. This ultimately results in system instability or crashes. In extreme cases, the Head node may become unresponsive, requiring manual intervention to restore functionality.

Software management

This section provides instructions on how to install and manage software on the cluster.
Anaconda, Spack, and Singularity have already been installed in the /users directory and are ready for the user to utilize.

Anaconda (Conda)

Conda helps set up virtual environments with the latest packages for the following:

- Anaconda 3
- R version 4
- AWS CLI version 2 (On the head node only)
- Go
- Singularity
- Spack
- Splunk Agent
- TKI
- MFA
- Cuda12
- Mountpoint for Amazon S3
- CrowdStrike (Nessus agent and Falcon Sensor are on the head node only)

- MySQL client (on the head node only)
- Python, TensorFlow, PyTorch, Apache MXNet, Nvidia Driver, Nvidia CUDA11 stack, EFA installer,

Environments:

• Check on Conda configuration and information:

```
$ conda config -show

$ conda clean --all

$ conda info

$ conda update -n base conda
```

• List virtual environments:

```
$ conda env list
```

• Create a virtual environment:

```
$ conda create -n python38

$ conda env create –n python38 -f /path/to/environment.yml
$ conda env create –n python38 -f /path/to/environment.yml -
force
$ conda env create –n python38 -f /path/to/environment.yml -d
$ conda env create –n python310 python=3.10
python38 and python310 are examples
```

• Activate a virtual environment:

```
$ conda init bash
$ source .bashrc
$ conda activate python38
```

• Save a virtual environment:

```
$ conda env export -n python38 -f python38_environment.yml
$ conda env export -n python38 > python38_environment.yml
```

• Deactivate a virtual environment:

```
$ conda deactivate
```

• Delete a virtual environment:

```
$ conda env remove -n python38
```

Packages in Environments:

- **Check on packages:**

```
$ conda list
```

- **Search for packages:**

```
$ conda search r-base
```

- **Install packages:**

```
$ conda install r-
base=3.6.0 $ conda
install r-base $ conda
install r-essentials
```

- **Update packages:**

```
$ conda list|grep r-base
$ conda search r-base
$ conda install r-base=4.2.0
$ conda list|grep r-base $ conda update --all
```

- **Remove packages:**

```
$ conda remove r-base
$ conda uninstall r-base
```

Using Local Storage

Some compute nodes offer non-volatile memory express (NVMe) solid state drives (SSD) volumes, which provides faster and larger temporary data storage while running a computation job.

To utilize this local storage on your node, follow the steps below:

```
#!/bin/bash

#SBATCH -N1

#SBATCH -n1

#SBATCH -J mnist

#SBATCH --output=%x_%j.out

#SBATCH --mail-user [YOUR_EMAIL]

#SBATCH --mail-type BEGIN

#SBATCH --mail-type END

#SBATCH --mail-type FAIL

#SBATCH --partition=c96-m768

#SBATCH --account=general

#SBATCH --mem=2G


conda init bash >/dev/null 2>&1 source

~/.bashrc

conda activate tensorflow_python39


sbcast /scratch/[YOUR_DIRECTORY]/inputdata.txt
/localscratch/[YOUR_NETIDDIRECTORY]/$SLURM_JOB_ID/inputdata.txt


python YOUR_PYTHON_PROG.py
/localscratch/[YOUR_NETIDDIRECTORY]/$SLURM_JOB_ID/inputdata.txt \
                        >>
/localscratch/[YOUR_DIRECTORY]/$SLURM_JOB_ID/outputdata.txt
```

This functionality is provided only in selected partitions. Please refer to the cluster banner for details.

All users of the compute node will have access to data stored here while the job is running.

Copy data there before beginning the computation job and then copy results back to your /users, /group or / scratch / local storage. Data is only available when the job is running and will be purged after the job.

Spack

What is SPACK?

Spack is a package manager that can be used to build, query, install, and manage packages on a variety of platforms. It is a tool designed to support multiple versions and configurations of software on a wide variety of platforms and environments. It was designed for large supercomputing centers, where many users and application teams share common installations of software on clusters with exotic architectures,

using libraries that do not have a standard ABI. Spack is non-destructive: installing a new version does not break existing installations, so, many configurations can coexist on the same system. More information on SPACK can be found Here.

Why use SPACK?

Users, programmers, and administrators of HPC can build a package with multiple versions, configurations, platforms, and compilers. It can be used to all programming languages like Python, C, C++, Fortran, R, etc. All versions of those programming languages can coexist on the same server. SPACK can also manage virtual environments in HPC like Conda and Python venv

• Installation of SPACK

```
$ git clone -c feature.manyFiles=true
https://github.com/spack/spack.git
~/spack $ git checkout releases/v0.18
$ . share/spack/setup-env.sh
```

• Verification

```
$ spack -V
```

Basic usages of SPACK

- **Check available packages**

```
$ spack list
```

- **Check available packages for Python**

```
$ spack list 'py-*' or R
$ spack list 'R-*'
```

- **Check package and select a version**

```
$ spack versions zlib ==> Safe
versions (already checksummed):
1.2.12 1.2.11 1.2.8 1.2.3
```

```
$ spack install zlib@1.2.8
```

- **Check on installed packages**

```
$ spack find
```

- **Check on compilers**

```
$ spack compilers
```

Create and activate virtual environments:

- **Check virtual environments**

```
$ spack env list
```

- **Create a virtual environment called myproject**

```
$ spack env create myproject
$ spack env list
$ spack env activate myproject
```

- **Check packages in the myproject virtual environment**

```
$ spack find
$ spack install tcl
$ spack find
```

- **Deactivate the myproject virtual environment**

```
$ despacktivate
```

- **Delete the myproject virtual environment**

```
$ spack env remove myproject
```

Singularity

What is Singularity?

Singularity is a container platform for HPC.

References: https://docs.sylabs.io/guides/3.10/user-guide/

https://docs.sylabs.io/guides/3.10/admin-guide/

Why use Singularity?

- Singularity allows you to create an isolated environment which contains software in a way that is portable and reproducible.
- Singularity can build a container, which can be executed on different platforms.

Installation of Singularity

Reference document

Two types of formats of Singularity images:

1. A Singularity Image Format (SIF) file

1-1 Compressed and read-only (Small and unmodifiable)

1-2 Exists as a file.

2. Sandbox directory format

2-1 Uncompressed and modifiable 2-2. Exits as a directory.

3. A SIF and sandbox can be easily converted to each other.

Sources of base image for Singularity containers

- docker://
- library://
- shub://

Build a Singularity image

1. vi ubuntu.dif

```
Bootstrap: docker
From: ubuntu:18.04


%post
  apt-get -y update
  apt-get -y install python


%files
  hello_world.py /


%runscript
  python /hello_world.py
```

2. $ sudo singularity build ubuntu.simg ubuntu.dif

Use a Singularity container

```
$ ./ubuntu.simg
Or
$ singularity run ubuntu.simg
```

Inspect a Singularity image

```
$ sudo singularity inspect --deffile ubuntu.simg
```

For additional information refer to this user guide.

For installing any additional applications or packages please reach out to the support team at hpc.help@emory.edu

MATLAB Runtime

On user's request in our latest upgrade, we included MATLAB Runtime in the cluster. The MATLAB runtime can be found at path.

```
/opt/MATLAB/MATLAB_Runtime/R2023b
```

MATLAB Runtime is included so that users can get their executable files and get advantage of MATLAB runtime to submit their jobs and run on compute nodes.

(**Note:** Users should already have access to a MATLAB compiler license. This way, they can use their own MATLAB compiler to compile MATLAB scripts and then transfer the executable and bash scripts to the cluster to run using SLURM scripts.

The following examples will show you how a MATLAB runtime works.

Example1:

Step1:
Save the following MATLAB script as *hello.m*

```
function foo =
echo(arg)
fprintf('Hello
World! %s\n', arg);
  foo = 0; end
```

Step2:
Compile the matlab script *hello.m*

```
mcc -m hello.m
```

Step3:
After compiling the hello.m script it will multiple scripts along with executable.

```
hello
includedSupportPackages.txt    readme.txt
run_hello.sh hello.m
mccExcludedFiles.log
requiredMCRProducts.txt
unresolvedSymbols.txt
```

Step4:
copy those files to the Hyperc3 cluster using *scp* or your preferred method

(**Note**: You just need to copy the executable and .sh files. In the above case hello, run_hello.sh files)

Step5:
Provide executable permission to run_hello.sh file, hello executable

```
chmod +x hello
chmod +x
run_hello.sh
```

Step6:
Use a slurm script to run the matlab executable on compute nodes

```
#!/bin/bash
#SBATCH --job-name=matlab_job        # Name of the job
#SBATCH --nodes=1                       # Number of nodes
requested
#SBATCH --ntasks-per-node=1  #SBATCH --
output=%x %j.out  # Name of the output file
#SBATCH --error=%x_%j.err
#SBATCH --mem=1G                     #Amount of memory needed
sh run_hello.sh /opt/MATLAB/MATLAB_Runtime/R2023b prasad
```

In the above slurm script in the last step we are executing the .sh script and also passing the input to the matlab executable

Example 2:
Compile the following test.m script

```
function result =
parallel_calculation(n_str, num_cores_str)
% Convert input arguments from strings to
integers     n
= str2double(n_str);
    num_cores = str2double(num_cores_str);

    % Check if conversion was
successful     if
isnan(n) || isnan(num_cores)
        error('Invalid input: n and num_cores must be
integers.');
```

```
    end
    % Create a parallel pool with the desired number of workers
parpool('local', num_cores);

    % Define the data or operation to be
parallelized     data = 1:n;

    % Perform some operation in parallel (example: squaring each
element)     result = zeros(size(data));  % Initialize result
array     parfor i = 1:numel(data)
        result(i) = data(i)^2;  % Example operation
(squared)     end

    % Display the result
disp(result);

    % Close the parallel pool
delete(gcp('nocreate')); end
```

After the compilation follow the same steps as in example 1 above.

Submit the job using slurm script:

```
#!/bin/bash
#SBATCH --job-name=matlab_parallel
#SBATCH --output=%x_%j.out
#SBATCH --error=%x_%j.err
#SBATCH --nodes=1
#SBATCH --mem=4G
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=4
#SBATCH --time=04:00:00
#SBATCH --partition=c64-m512

# Run MATLAB script
sh run_test.sh /opt/MATLAB/MATLAB_Runtime/R2023b 100 4
```

In the above slurm script one task/process needs 4 cpu cores. So, our operation of calculating squares will be split between 4 cores.
In the last line of above script, we are passing value of n and num of cores to the run script as arguments.

Terms and Policies

Rules of behavior

Hyper C3 users are expected to operate with the understanding that they have nontrivial access to stage software and data and execute compute processes within a large, shared environment and that they must follow all HyPER C3 Rules of Behavior and agree that they will strive to operate in a careful and secure manner.

Users must agree to the Cluster's full Rules of Behavior here.

To protect your data in a shared environment, refer to these policies and examples (more in full policy document listed above).

- Users should not share data of any kind between two **/group** sub-directories owned by different Sponsor users without prior approval from both Sponsors. For example, **jjones2** is affiliated to work for **jsmith1** and **scarter3**. Therefore, **jjones2** should not transfer files between the sub-directory of /group/**jsmith1**g00 and /group/**scarter3**-g00 without prior written approval from both **jsmith1** and **scarter3**.

- Cluster users must not change the file permission rights of directories or files in a way that would introduce more permissive or global access (such as "world" or "other").
  For example, commands like chmod o+r, chmod o+rx, chmod o+rwx or any other variation that includes other rights should not be applied, even if the system allows such changes.

- Users of the cluster service must not attempt to hack, disable, bypass, or interfere with security controls, restrictions, or monitoring mechanisms.

- The default "umask" has been setup to enable ease of use in the /group subdirectories by the Sponsor User and related Affiliate Users. Most users don't need to understand this, but if you do want to change the umask for some reason, the umask's first digit must remain 0 and last digit must stay 7. You can use other umask values as desired as long as they fit the pattern of 0xy7 where x and y are valid digits from 0 to 7.

Compute Policy

GPU requirements: Users must specify the number of GPUs, CPU cores, and memory required for their jobs. This information is used by Slurm to schedule jobs and ensure that they do not exceed the available resources.

Default partition: If a user does not specify a partition, their job will be submitted to the default partition "c64-m512". This partition is a general-purpose partition that is suitable for most jobs.

GPU partition: The GPU partition is only available for jobs that require GPUs. Jobs submitted to this partition will be scheduled on nodes that have GPUs available. Jobs that require GPUs cannot be submitted to non-GPU partitions.

GPU hours: The maximum number of GPU hours per user is 500 for A100, L40s partitions. This limit is in place to ensure that all users have fair access to the GPU resources.

Head node: The head node is the main node of the cluster. It is not intended for running jobs and can only be accessed by cluster admins.

LoginNode: The login node is the node where cluster users will be logged into and submit jobs from.

Fairshare Policy:
Resource Allocation:

Fairshare ensures that each user or group gets a fair share of the available resources (such as CPU time, memory, or nodes) over a defined period, regardless of the instantaneous demand.

Accumulation of Shares:
Fairshare calculates usage over time and assigns "shares" to users or groups based on their historical resource consumption. Users or groups who have utilized fewer resources recently will have a higher share of resources in the future, while those who have used more will have a lower share.

Resource Allocation Decision:
When allocating resources for pending jobs, SLURM takes into account FairShare information along with other factors like job priority and resource availability. Jobs submitted by users or groups with higher FairShare shares are more likely to be allocated resources.

Adaptation over Time:
Fairshare continuously adjusts based on users' or groups' resource usage patterns, ensuring that resource allocation remains fair over time.

For e.g.:
If a user for instance has 10 jobs running, when you submit your job, it will also be running without affecting others. That is, if a user submits 20 jobs and 4 are running using all resources in a queue, if you submit your 4 jobs in that same queue requesting all the resources, your job get the priority and should run before the users other 16 jobs.

Storage Policy
Please refer to Storage Quotas.
User Account Policy
Please see the User Account Expiration for more information on user accounts.

Submitting Jobs via Open OnDemand

We have integrated Open OnDemand, a web-based portal that allows users to access the cluster and submit jobs directly from their browser without needing to use the command line.

Accessing Open OnDemand
- Open your web browser and go to:

https://ondemand.it.emory.edu (or) **https://cirrostratus.it.emory.edu**
- Log in using your Emory credentials.

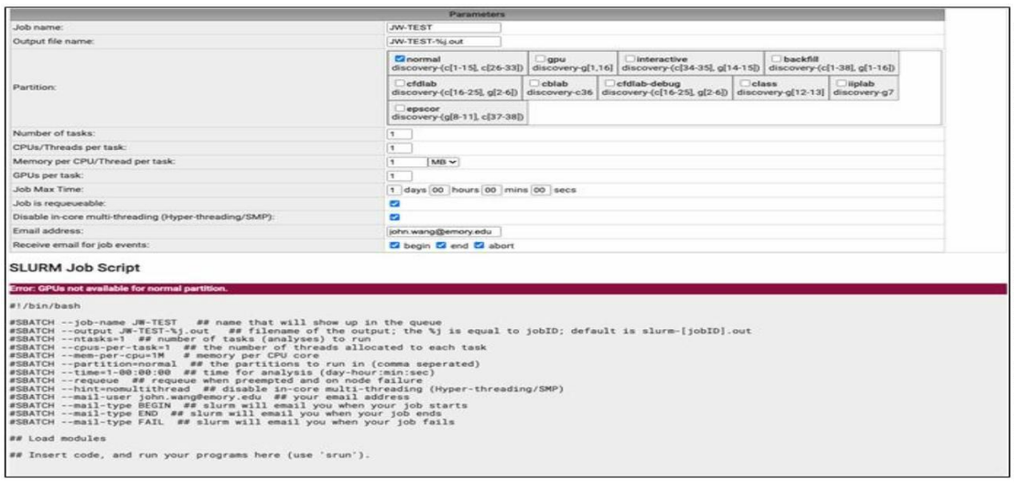
More instructions on using Open OnDemand can be found here

Help and Support
This section provides instructions on how to get help with the cluster.
Reach out to the support team at hpc.help@emory.edu for help.

Appendix

Generate SLURM Job Script

There are some great tools available online that can help generate SLURM job scripts. Beginners may be able to take advantage of these free tools to generate scripts with required options and parameters. One of these free tools is hosted by New Mexico State University and can be accessed here: https://hpc.nmsu.edu/home/tools/slurm-script-generator/generator/



Review Job Progress

After submitting the job, if users want to view the progress of the execution of the job, they may use the following commands. This will help them to *$ screen* – This command starts a new session.
$ run your job – Run your job in this new session.
Ctrl + a + d – Leave the session to run other commands. *$ screen -r* – return to where you left in this session.